

**Hex Grid Automata
Design Notes
(draft 01)**

Introduction

Hex grid automata (HGA) that is under the current visual studio project name VW_Automata is an application to perform defined cellular automata rules in a 2D hexagonal grid environment.

This can be considered as a complete and functional first step application that can perform some basic neighbour driven cellular automata and be expanded or forked for possible more complex automata rules and conditions.

The design of the code of this application is such that this will form a basis for a regular 2D Cartesian grid structure and both a 3D Cartesian and hexagonal grid structures.

The design incorporates that the user defines rule conditions for the cellular automata as a series of c/c++ like conditional statements that query the neighbours of a grid cell of a current hexagonal grid as well as itself. If any of a series of conditions are true, a value to the current processing step of the hexagonal grid, the grid cell is assigned a value according to which set of conditions are met as defined by the user.

A standard cellular automata step procedure.

The application uses ImGui and ImPlot as its major third party dependencies as well the usual dependencies they require and, and is coded in ISO C++20 Standard as a visual studio community project.

All these files are included to insure no breaking of the application code will occur due any updates as has happend in the past while coding or compiling other downloaded projects.

In future the name of this application will be changed when a suitable name is decided upon.

A user manual and code documentation are not yet provided, but will be included in a future update.

Dependencies

These are a list of the current third party dependencies for this project

- glfw
- glew
- ImGui
- imgui-docking
- implot
- tinyfiledialogs

Dependency source header files to be defined where \$(ProjectDir) is the project directory where the header files are located

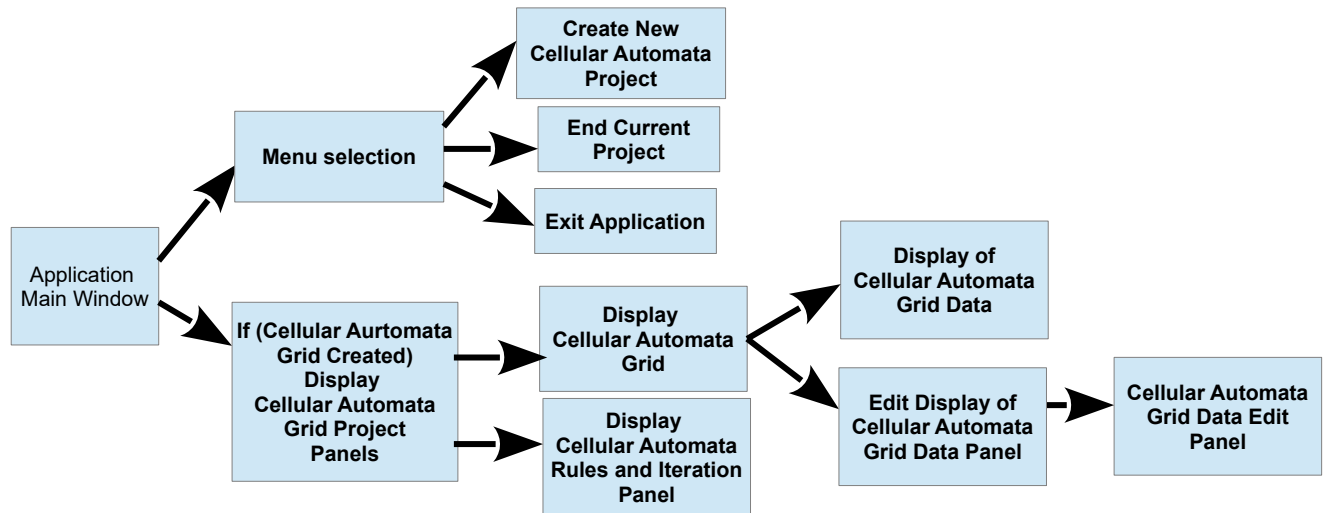
```
$(ProjectDir);
$(ProjectDir)Source;
$(ProjectDir)ThirdParty;
$(ProjectDir)ThirdParty\ImGui\imgui_docking;
$(ProjectDir)ThirdParty\ImGui\imgui_docking\backends;
$(ProjectDir)ThirdParty\ImGui\implot;
$(ProjectDir)ThirdParty\glew\include;
$(ProjectDir)ThirdParty\glew\src;
$(ProjectDir)ThirdParty\glfw\include;
$(ProjectDir)ThirdParty\glfw\src;
$(ProjectDir)ThirdParty\glm-1.0.1;
$(ProjectDir)ThirdParty\ImGradientHDR\src;
```

Dependency libs to be defined where \$(ProjectDir) is the project directory where the lib files are located

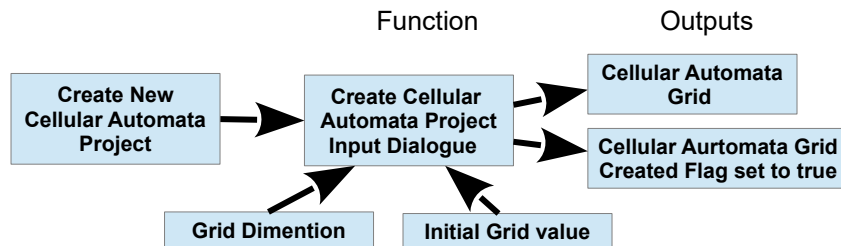
- opengl32.lib

1 Basic Application Design

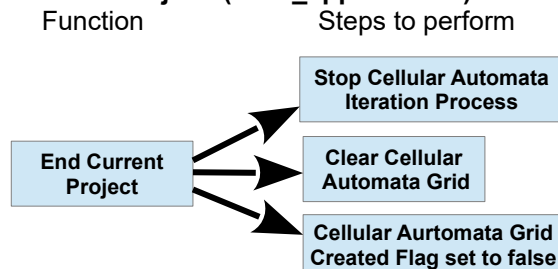
1.0 : Main application window (ahex_applicaion.h)



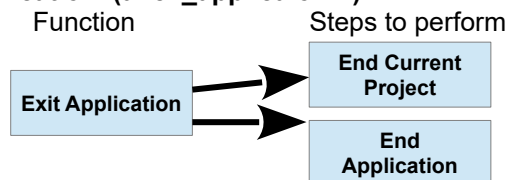
1.1 : Create New Cellular Automata Project (ahex_applicaion.h)



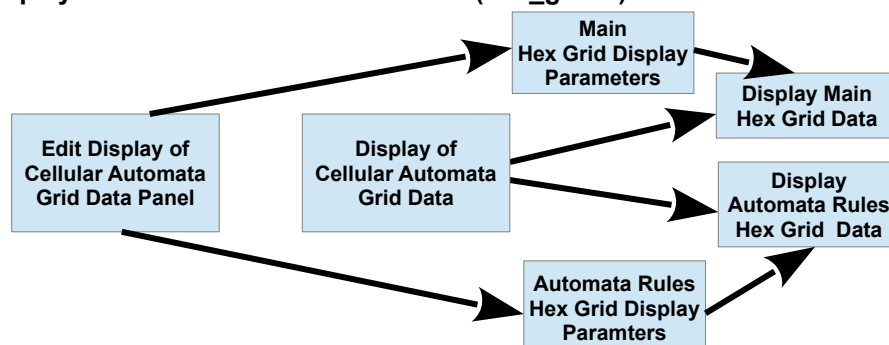
1.2 : End Current Project (ahex_applicaion.h)



1.3 : Exit Application (ahex_applicaion.h)



1.4 : Display of Cellular Automata Grid Data (hex_grid.h)



Use `ImPlot::PlotScatter` function to display the hexagonal cellular automata grid as a scatter plot with every

hexagonal grid cell assigned a x-y coordinate according to its index location and the hexagonal grid x-y dimension. Thus the scatter plot can have a display of a regular hexagonal grid lattice.

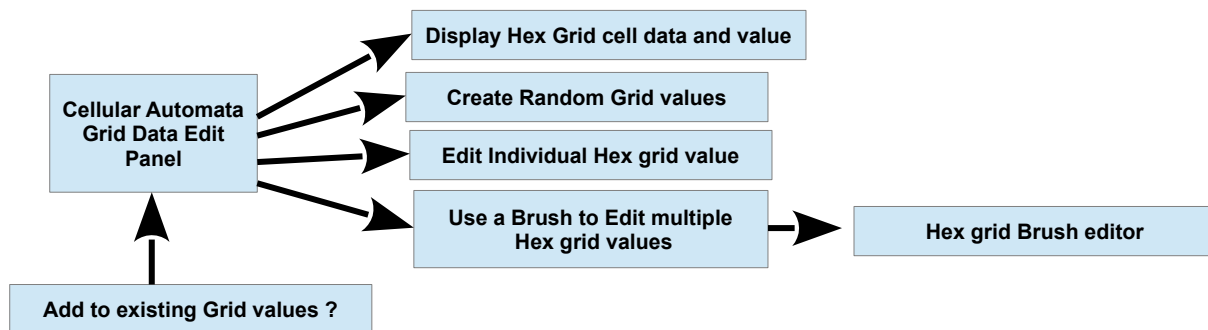
ImPlot::PlotScatter accepts an array of x-y coordinate pairs that can be in any order and of any array size. This means that ImPlot::PlotScatter can be used to plot representations of the current hexagonal grid rules as separately defined data sets of partial hexagonal grid data for each hexagonal rule and overlay on top of the main hexagonal automata grid display. Such hexagonal rules data uses a different structure/class to that of the main hexagonal grid that stores the cellular automata data.

The end display the ImPlot::PlotScatter function generates allows the user to interact with the display and return the x-y Cartesian coordinates that the mouse cursor is currently located at. The interactions are to change the display size, zoom in and out and pan the display. Changing the size of the display window generally distorts the aspect ratio of the display, and thus a method was introduced to change the display window size if a resize occurs to keep this aspect ratio constant and correct.

1.5 : Edit Display of Cellular Automata Grid Data (hex_grid.h)

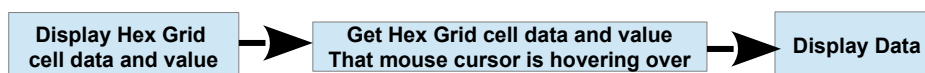
ImPlot::PlotScatter function has parameters to display the component points with a defined color, shape, and shape size that can be assigned as a variable value for each data array that is to be displayed. This is implemented by changing the variable value interactively in a window widget panel that then changes the display of the hexagonal grid appearance.

1.6 : Edit Cellular Automata Grid Data (hex_grid.h)



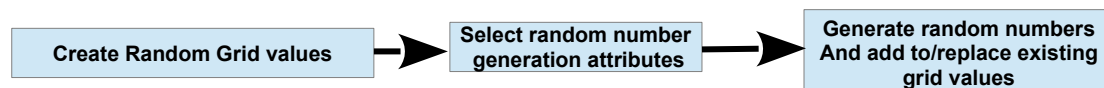
In order to define a cellular automata grid to be used in any cellular automata iteration process, a method needs to be available for the user to initialise and/or edit the current cellular automata grid. I decided that the window widget to do this function would only be displayed upon the user requesting for it to be displayed and not opened by default by the application. Thus within the Edit Display Cellular Automata Grid Data Panel was an activation checkbox placed to display such a window widget panel.

1.6.1 : Display data of hex grid that mouse is hovering over



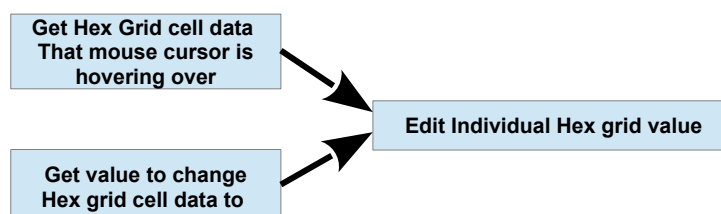
Within this panel is a display of the current Hex grid cell data that the user has a mouse cursor placed over within the display of the cellular automata grid data (1.4) for the user to inspect the hexagonal grid cell coordinates and grid cell data value.

1.6.2 : Generate random numbers to assign to each hex grid cell

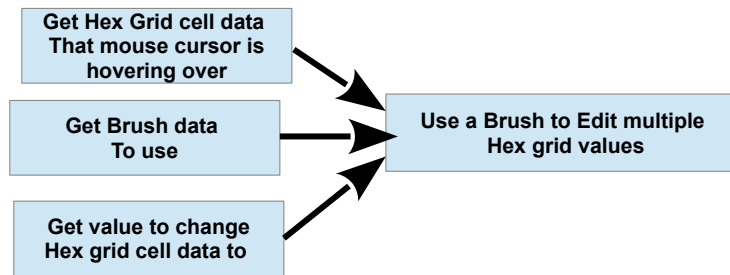


The creation of a random grid values utilises the C++ random library and has been designed to interactively select the parameters and generate the random numbers based upon the input requirements of the Random C++ library.

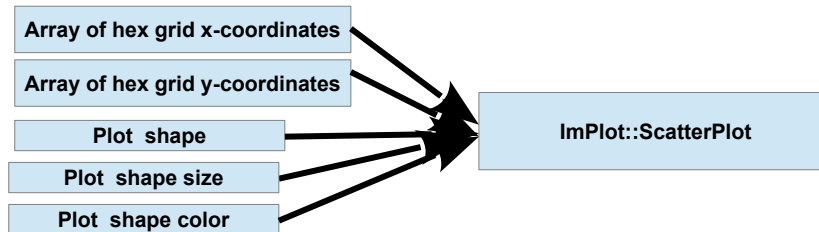
1.6.3 : Edit hex grid cell



1.6.4 : Edit hex grid with hex grid brush



1.7 : Display Hex grid using ImPlot scatter plot



2 Hexagonal Grid Base Class (hex_grid_base.h)

The **hex_grid_base_class** is the C++ class that stores the hexagonal automata grid data, data that defines the hexagonal automata grid and the functions to manage the and deletion of the hexagonal automata grid data.

To anticipate that this base class may have more than one datatype to use in any hexagonal automata application, it was decided to define the datatype of the hexagonal automata grid as a template.

2.1 : Dynamic C++ std::vector array

Hexagonal grid is stored as a single C++ dynamic vector array `std::vector` that can be constantly updated without the need for creation or deletion. That is a single vector array is defined as a variable when a project is created and is used to store, modify, and change size as the application requires.

To enable the hexagonal grid to be able to accept different data types, the decision was made to have the vector array assigned as a template variable. This then restricted the coding of the project where this template data type is referenced to be within C++ header files only.

Hexagonal grid has coordinate and colour data also stored as separate C++ vector arrays to be used to display the hexagonal automata grid using `ImPlot` as a scatter plot.

Because using `ImPlot` has been chosen as the main 2D display, it requires coordinate and color data to be stored as separate data arrays and thus have them as the same in the hexagonal grid base class.

Because it is easier to reference, the above hexagonal grid cell value, coordinate and colour data is not defined as a single datatype structure that is stored as a C++ vector.

2.2 : Hexagonal Grid definition

The Hexagonal grid is defined by its dimension and first grid cell coordinate in both global Cartesian and index of a possible parent hexagonal grid coordinates.

The correct dimension is vital for conversions to and from hexagonal index, x-y index and Cartesian coordinates.

When utilising Cartesian coordinates, the global Cartesian coordinate of the first grid cell is thus also vital as it gives a reference to the global Cartesian coordinates that the hexagonal grid exists within.

2.3 : Hexagonal Grid Functions

The Functions relating to the storage and retrieval of 2D hexagonal grid data is a modification an adaptation of the 3D **Trapezo-rhombic dodecahedron** voxel data storage system used in the Virtual Worlds project, and is to be used in the 3D version of this application. Some additional functions particular to this project have been added.

Purpose of the functions are to create and delete hexagonal grid, coordinate and color data. These functions are complicated if the hexagonal grid data in **2.1** was stored in a C++ structure data, and another reason for not to do this. The main purpose of the functions are to perform the task of coordinate translation from ant to hexagonal grid 1D index value to hex grid 2D index and Cartesian coordinates for a derived class to use to assign and modify hexagonal grid values.

2.4 : Hexagonal Virtual Grid Functions

Three virtual functions that must feature in any derivative class which will be different for each datatype used have been declared.

```
virtual void delete_hex_grid_data(hex_surface_index_data_type i) = 0;  
virtual void invalidate_hex_grid_data_value(hex_surface_index_data_type index) = 0;  
virtual bool display_hex_grid() = 0;
```

Any derivative class must have a function defined for each of these functions that is applicable to the datatype of the hexagonal automata grid cell that is being used.

3 Hexagonal Grid Cell Structure Data Type (hex_grid_base.h)

```
hex_grid_cell_data_struct_type

Hex grid index array
X-coordinate array
Y-Coordinate array

Hex grid display shape
Hex grid display shape size
Hex grid display shape colour

Functions::
Add hex grid cell coordinate/index data
Clear hex grid cell coordinate/index data
Display hex grid coordinate data
```

The hexagonal grid cell structure datatype (**hex_grid_cell_data_struct_type**) is a C++ structure datatype that is to be used in the display of hexagonal automata grid related data, but has no hexagonal automata grid cell data values. Therefore no template datatype is used or present in this structure. The primary purpose of the `hex_grid_cell_data_struct_type` is to define a list of hexagonal grid cells and coordinates that is a subset of a particular type or class of data of the main hexagonal automata grid to be displayed and over layed on the main hexagonal automata grid.

3.1 : Storage

```
std::vector of type integer to store hexagonal grid index values
std::vector of type float to store hexagonal grid x - coordinate values
std::vector of type float to store hexagonal grid y- coordinate values

shape variable to display hex grid cell representations
shape size variable of hex grid cell shape
colour variable of colour of hex grid cell
```

The data stored in this structure is a series of lists of the index of hexagonal grid cells and associated coordinates, colour and other graphical data that is used by ImPlot to plot to display hexagonal grid related data.

Because it is easier to reference, the above hexagonal grid cell index and coordinate data is not defined as a single datatype structure that is stored as a C++ vector.

These lists are stored as C++ dynamic vector arrays for each type of data and have the order or index of each list entry related to each other, and thus the management of any one vector array requires all others be managed as well.

This datatype has the one same shape and colour common for all hexagonal grid cells stored, and therefore only one variable instance of colour, shape and shape size data is required to be defined.

3.2 : Functions

Functions of the `hex_grid_cell_data_struct_type` are to manage the data and display the sub-set of hexagonal automata grid data.

3.2.1 : Add hexagonal grid index and coordinate data.

```
add_hex_grid_cell (hex_grid_index,x_coord,y_coord)
    Add index to hex grid cell index array
    Add x-coord to hex grid cell x-coord array
    Add y-coord to hex grid cell y-coord array
```

3.2.2 : Clear hexagonal grid index and coordinate data.

```
Clear hex grid data
    clear and shrink to fit hex grid cell index array
    clear and shrink to fit hex grid cell x-coord array
    clear and shrink to fit hex grid cell y-coord array
```

3.2.2 : Display hexagonal grid data utilising ImPlot scatter plot function.

Display the hexagonal grid data defined as C++ vectors and variables as in **3.1** using ImPlot scatter plot function.

See **Fig 1.7** for reference

4 Hexagonal Sub Grid Cell Structure Data Type (hex_grid_base.h)

The hexagonal sub grid structure data (**hex_sub_grid_struct_type**) is an experimental datatype devised to define a hexagonal automata sub grid that can be used for processing a sub set of the main hexagonal grid, or to used to perform grid to grid operations such as boolean like functions such as merging of grids. Even though testing of this concept worked, it is thus far not used in the main application, but remains as a future prospect.

The sub grid is defined basically as the lower left hexagonal grid index and upper right hexagonal grid index of the parent hexagonal grid it is a sub grid of.

For display purposes, because this structure utilises the ImPlot scatter plot function, the coordinates of the hexagonal sub grid cells are stored in two C++ vectors for the x and y coordinates.

4.1 : Functions

Functions to validate the boundary conditions in regards to another hexagonal grid and see if a given coordinate exists within the sub grid.

Function to display the hexagonal sub grid utilising the ImPlot scatter plot function.

5 Hexagonal Sub Grid Manager Class (hex_sub_grid_manager.h)

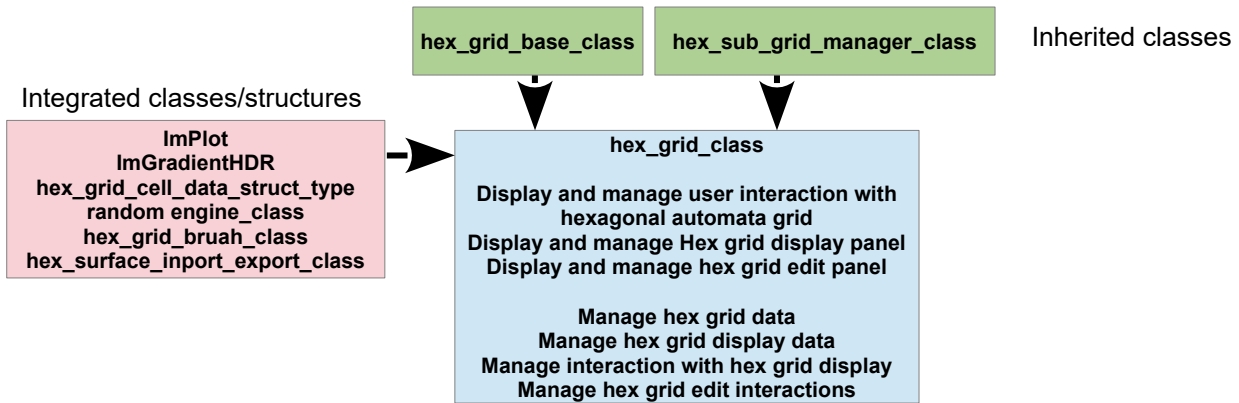
The **hex_sub_grid_manager_class** is the C++ class that is created and define to perform tasks involving the **hex_sub_grid_struct_type**. It is an experimental set of functions that accept two variables of the data structure datatype **hex_sub_grid_struct_type** and perform such tasks as merging the two grids or performing tests to see if one grid A overlaps with the other grid B. It is not currently used but is retained for possible future use and implementation.

6 Hexagonal Automata Grid Class (hex_grid.h)

The **hex_grid_class** is the C++ class that is created and define to store and manage the application hexagonal automata grid data, and to perform the tasks to display the user interface and user interactions that modify the hexagonal automata data and the display of that data.

The **hex_grid_class** is a derived class of the **hex_grid_base_class** and **hex_sub_grid_manager_class**. Even though at present the **hex_sub_grid_manager_class** functions are not used in the application, this class is preserved for possible future use and utilisation.

The hexagonal automata grid class is referenced throughout the application and is the main and central foundation to its functionality



6.0 Display and manage hexagonal grid display and user interaction with it

The display of the main hexagonal grid and user interaction with it are performed and managed largely by the `ImPlot::ScatterPlot` function. `ImPlot` has many advantages to enable user interaction and display of data.

The `ImPlot::ScatterPlot` function has

1. Inbuilt user interactions for the tracking and display of a mouse cursor and the Cartesian coordinates in which the mouse cursor is located.
2. Ability to zoom in and out of display.
3. Ability to pan display.
4. Ability to resize the display
5. Ability to customise to a degree the display

When applying 4, it was found that `ImPlot` could not maintain a correct aspect ratio of displaying the hexagonal grid structure and thus a method was found to modify the window display size and keep the correct aspect ratio whenever one of the sides of the display was changed.

From 1 a Cartesian coordinate of the current cursor location within the hexagonal grid could be extracted for editing the hexagonal grid data or querying of hexagonal grid values. Done by `hex_grid_base_class` function to translate or convert Cartesian coordinate to hex grid index or hex grid array coordinates.

6.0.1 Display hexagonal automata grid data sub sets

Various hexagonal automata grid subsets are defined to display sets of hexagonal automata data with a common data value. These subsets are of the `hex_grid_cell_data_struct_type` datatype (section 3) and are stored in a C++ vector dynamic array `hex_grid_cells_data_list`. That is

```
std::vector<hex_grid_cell_data_struct_type> hex_grid_cells_data_list
```

The `hex_grid_cell_data_struct_type` has its own `ImPlot::ScatterPlot` display function and for each member that exists within `hex_grid_cells_data_list`, it is displayed according to the coordinate and display data defined for that subset of hexagonal automata grid values.

Mostly used for the display of automata rule results.

6.1 Display and manage hexagonal grid display panel

Display of hexagonal automata grid uses the `ImPlot::ScatterPlot` function See **Fig 1.7** for reference.

6.1.1 Display of hexagonal automata grid cells

This is to display the hexagonal grid cells, not a reference to the values they may have been assigned A `ImGui` window widget is defined with appropriate display widgets and variables to interactively modify the scatter plot representing a hexagonal grid display. These being in pseudo code

Plot colour, plot shape and plot size.

6.1.2 Define Range of coordinates to display

The `ImPlot:ScatterPlot` function also can have limits or range applied to the axis coordinate values that are displayed. Thus `ImGui` widgets are defined with appropriate variables to interactively modify the scatter plot range of the hexagonal grid display. These being in pseudo code

```
minimum x-coordinate, maximum x-coordinate  
minimum y-coordinate, maximum y-coordinate
```

`ImPlot` incorporates a function `SetupAxisLimitsConstraints` which accepts minimum, maximum limits of the x or y axis to display.

6.1.3: Display hexagonal grid cell data values.

The hexagonal automata grid cell values have the colour of each hexagonal grid cell defined from a colour gradient. This colour gradient is defined and constructed by a modified version of the `ImGui ImGradientHDR` extension widget. By defining a minimum and maximum value to the limits of this gradient, a colour can be calculated from this gradient based upon an extrapolation from the minimum to maximum value and the location of the gradient colour markers that define the colours of this gradient.

An `ImGui` window widget for the user to enter the minimum and maximum values of the gradient limit is defined with appropriate variables. The modified `ImGradientHDR` widget is displayed which incorporates the ability of the user to save and load gradients that the user has created.

Colour data for each hexagonal grid cell is stored in a C++ `std::vector<ImU32>` `hex_colors` dynamic array.

What this means is that any interactive updates of colour data can possibly create very large lag times if a gradient colour marker is moved or changes colour. For this reason the hexagonal automata grid cell value colour data is updated only when the user activates the provided update hex grid colors button widget.

6.2 Display and manage hexagonal grid edit panel

The hexagonal grid edit panel is not displayed by default, but is displayed upon activation by the user. It consists of four parts.

User interaction with the hexagonal automata grid when in edit mode is restricted in that pressing left mouse button will assign a value to a hexagonal automata grid cell and not enable the user to directly pan or zoom. To pan or zoom, the user will need to press the shift key before the left mouse button.

6.2.1 Display hexagonal automata grid cell data

When the user interacts with the hexagonal automata grid display (section 6.0), the hexagonal automata grid cell data that the mouse cursor is hovering over is extracted from the main hexagonal automata grid vector array and displayed. A hexagonal grid coordinate, index and cell value are displayed for the user to examine in the top section of the editor `ImGui` window widget.

6.2.2 Random hex grid generation

The hexagonal automata grid can be modified as random numbers using the C++ random class. The C++ random class generates random numbers according to a common structure of simplified form

```
random number = number distribution(random number generator(seed))
```

There are several number of random number generators, and several number of number distribution algorithms where all distribution algorithms can accept all random number generators.

Every random number generator can accept one of two methods of seed values being a known constant, or some form of unknown machine time or random device based number.

Every number distribution algorithm has a different set of input parameter values to define its distribution pattern of numbers.

In the end there are

(number of random number generators) x (number of number distribution algorithms)
combinations of random number distributions that are possible.

What this means is that each random number generator can have any of three methods to define the seed value for generating random numbers, and each number distribution algorithm can have any one of the number of number generators as input.

End result is that all combinations of these factors has to be taken into account in defining the user interface

and interaction and the background random number generation and assignment to hexagonal automata grid cell values.

User definition of random number seed value beyond the default set value of the C++ compiler to use is done via ImGui checkbox widget which upon activation displays further options for a seed value to be defined.

Selection of random number generator and distribution performed using ImGui combo box widget.

The parameters for each number distribution is displayed depending upon which distribution was selected. Therefore there needs to be an ImGui set of widgets and function to display those widgets for each number distribution algorithm.

Once all these selections and parameters have been defined, the generation of random numbers and assigning them to each hexagonal automata grid cell is performed. To do this there needs to be a function for each number distribution algorithm and the function used depends upon which number distribution algorithm was selected.

The `random_engine_class` class handles all the tasks and processing of the inputs that the user has defined and assigned to be used by and within this class.

6.2.3 Edit hexagonal automata cell data value(s)

As stated in section 6.2.1 the user interaction with the main hexagonal automata grid display has the ability to extract the coordinates of the mouse cursor within that display and from that the current hexagonal automata grid cell that the mouse is hovering over can be found.

Two options to directly edit the hexagonal automata grid values exist.

1. User enters a value to edit individual single hexagonal grid cell value and to change the hexagonal automata grid cell that the mouse is hovering over presses the left mouse button to change the hexagonal automata grid cell to that value.
2. User activates the use hex grid brush to open the hex grid brush window widget. The user selects the brush to use and where the user then hovers over a hexagonal automata grid cell, that cell is the centre of the selected brush pattern and every hexagonal automata grid cell that falls within the pattern of the brush changes its value to the value that has been defined in 1.

1 is simple to implement by getting the hexagonal automata grid cell index value from the Cartesian coordinates that the mouse is hovering over and changing its value to the defined edit value.

In 2 the hexagonal brush is in essence a hexagonal grid defined within the `hex_grid_brush_class` as a C++ vector array just as it is in the `hex_grid_base_class`. Difference being that the hexagonal grid cells in this class are set to a value of one for the active part of the brush to change the hexagonal automata grid cell data, and zero for not to.

The display of the hexagonal brush window widget and handling of all the functionality of the widgets within the window widget is performed within the `hex_grid_brush_class`.

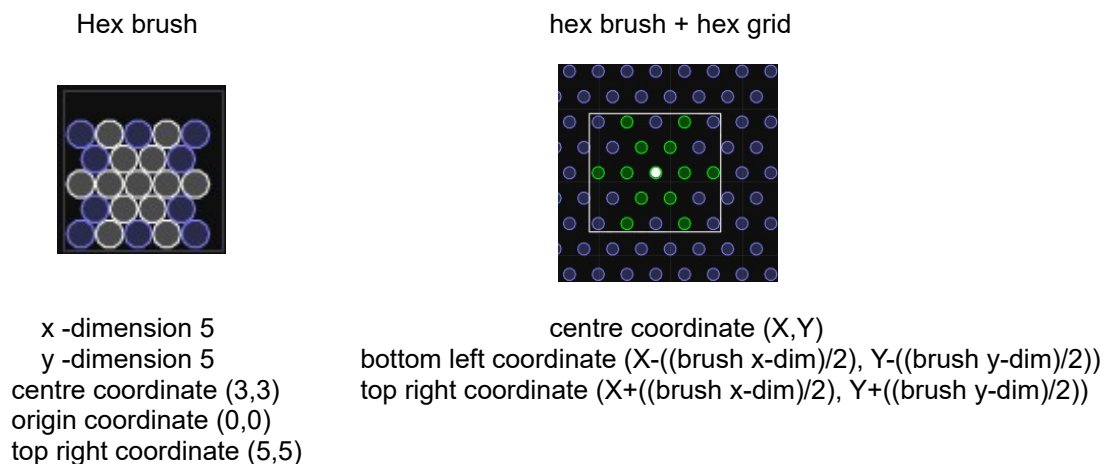


Fig 6.2.3

By extracting the hexagonal grid data from the `hex_grid_brush_class` class, the range of hexagonal automata grid cells to be change is calculated as a 2D hexagonal grid coordinates. By iterating over this coordinate range, the index of the hexagonal automata grid cells is found and changed according to the hexagonal brush grid that corresponds to be layered over that hexagonal automata grid cell and the edit value.

6.2.4 Edit hexagonal automata cell data value of border cells

In some instances, a cellular automata model may not be able to detect it is at the border or boundary of the hexagonal grid, and fail to perform an iteration process correctly leading to errors in the model. Thus a function and option has been added to assign the hexagonal grid boundary cells at the edge of the grid on all sides with a value nominated by the user so a model can be constructed to have rules that determine if a neighbour cell is a hexagonal border cell and act appropriately.

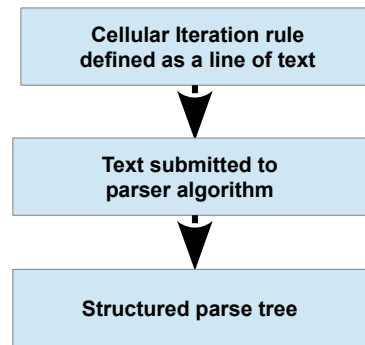
By activation of a ImGui button widget all the hexagonal automata grid cells that have a hexagonal grid coordinate that is on the border of the hexagonal automata grid is assigned the value as defined by the user in the appropriate ImGui widget and associated variable. These hexagonal automata grid cells have the coordinates of

(0,0) to (x-dimension,0) : (0,0) to (0,y-dimension)

(0,y-dimension) to (x-dimension, y-dimension) : (x-dimension,0) to (x-dimension, y-dimension)

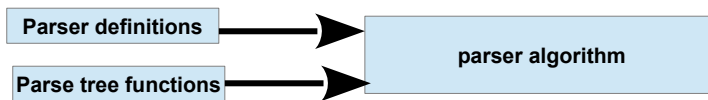
7 Cellular Automata Iteration Process

7.1 Create cellular automata rule as a parse tree



1. Cellular automata Iteration rule defined as a C/C++ like statement in a string of text
2. Pass rule in 1 as a string of text to a parser algorithm to test for validity of the syntax of the text and if valid, convert the text into a structured parse tree to be used for the execution of the cellular automata rule.

7.2 Parser algorithm (logic_parser.h)



Very simple and basic parser of top down type that has limited grammar and language capabilities to only be able to perform basic arithmetic and logic functions. No variables permitted but has a set of token definitions that can be used to retrieve the current hexagonal automata grid cell value and those of its neighbours.

7.2.1 Parser definitions

Parser definitions (**parser_definitions.h**) contains the sets of parser token definitions that are to be used in validating the submitted cellular automata rule text and creating a structured parse tree consisting of nodes of a C++ structure datatype.

Parser tokens divided into types logic, comparison and mathematical operators, data value and bracket or brace pairs.

Structured parse tree is a binary tree of nodes of a C++ structure that must have a function to evaluate the token operator it represents.

Bracket or brace pairs are used in the process of the construction of the structured parse tree to structure the parse tree into left and right substructures or leaves of the main operator. Eg $((a+b) * (c-d))$ has the expression $a+b$ as a sub tree on the left of the main operator $*$ token and $c-d$ as a sub tree on the right. These braces are not part of the structured parse tree, but are used in its construction. The parser algorithm requires that all operator declarations must be enclosed within two brace pairs or functionality of the resultant parser tree will result in an incorrect value.

Logic, comparison and mathematical operators are of two types unary and binary.

Data value is one of the datatype integer, float, boolean or double, and can be stored as a union so long as the stored datatype is known. However a not a number value (nan) is also permitted to be stored to indicate a hexagonal automata grid cell or neighbour reference is beyond the bounds of the hexagonal automata grid, or is in an undefined region.

This leads to an enumeration class for parser node functions being defined as

```
enum class function_type_enum { unary, binary, value, nan }
```

pairing this function type to the token text as

```
typedef std::pair<function_type_enum, std::string> function_token_definition_type
```

enables the parser tree node to have the information of what function type the token represents.

By defining a enumeration class for the permitted parser node data types as

```
enum class hex_grid_value_data_type_enum { Boolean, Float, Integer, Double, nan }
```

and having a union of data values defined as

```
union node_data_value {
    bool    bvalue;
    int     ivalue;
    float   fvalue;
    double  dvalue;
};
```

enables pairing of the data value of this union value to the data type of the value in that union as

```
typedef std::pair<parse_value_data_type_enum, node_data_value > node_data_value_type
```

Putting all this together, a base C++ structure datatype suitable to be used in a parser structured tree can be defined as

```
struct parser_base_node {

    function_token_definition_type function_token_definition;

    node_data_value_type value;

    bool not_a_number(node_data_value_type data_value) {
        if (data_value.first == parse_value_data_type_enum::nan)
            return true;
        else
            return false;
    }

    void define_value(parse_value_data_type_enum data_type, node_data_value data_value) {

        switch (data_type) {
            case(parse_value_data_type_enum::Boolean): value.second.bvalue = data_value.bvalue; break;
            case(parse_value_data_type_enum::Float):   value.second.fvalue = data_value.fvalue; break;
            case(parse_value_data_type_enum::Integer): value.second.ivalue = data_value.ivalue; break;
            case(parse_value_data_type_enum::Double):  value.second.dvalue = data_value.dvalue; break;
            case(parse_value_data_type_enum::nan):     value.second.ivalue = 0; break;
        }

        value.first = data_type;
    }

    virtual bool Evaluate() { return false; }
    virtual void PrettyPrint(std::ostream& writer) {};
    virtual ~parser_base_node() = default;
};
```

For a simple parser to process a limited range of functionality as basic arithmetic and logical evaluation, and returning a value, three types of parser tree nodes are needed.

literal_node : To store a value defined as a string of characters that was defined in the cellular automata rule

unary_node : To define a unary operator function to be performed on an expression represented as a parse tree node

binary_node: To define a binary operator function to be performed on a left and right expressions represented as a left and right parser tree nodes.

Defined in parser_definitions.h

```
class literal_node : public parser_base_node { // raw text representing a number
private:
    std::string _literal;

public:
    explicit literal_node(std::string literal) : _literal(std::move(literal)) {}
    std::string RoleName() const { return _literal; }

    void PrettyPrint(std::ostream& writer) override {
        writer << "(" << _literal << ")";
    }
};
```

```

};

struct unary_node : public parser_base_node {
    parser_base_node* _expression;

    unary_node(parser_base_node* expression) : _expression(std::move(expression)) {}

    parser_base_node* Expression() const { return _expression; }
};

struct binary_node : parser_base_node {
    parser_base_node* _leftExpression;
    parser_base_node* _rightExpression;

    binary_node(parser_base_node* leftExpression, parser_base_node* rightExpression)
        : _leftExpression(std::move(leftExpression)), _rightExpression(std::move(rightExpression)) {
    }

    parser_base_node* LeftExpression() const { return _leftExpression; }
    parser_base_node* RightExpression() const { return _rightExpression; }
};

```

7.2.1 Parser functions class

The parser functions class, `functions_class` defined in **functions.h** performs all of the parser tree structure unary, binary logical and mathematical operations as defined by each parser tree node assigning the result to the parser tree node.

If a parser token is designated for the retrieval of the hexagonal automata grid cell or neighbour data values is present, this is where the data value of that hexagonal automata grid cell is retrieved to be used in any parser tree operation.

In essence, only binary operators are used and only one function is need to perform all the binary operator functions.

This `perform_binary_operator_function` within the functions class is in essence, a series of if statements with similar coding to determine which binary operator is being processed, perform the operation of that operator and return a boolean value for conditional or logic operation.

For a binary mathematical function, perform the binary mathematical operation and assign the result to the parser tree node, and return a true value.

7.2.1.1 Not a number (nan) arithmetic

Throughout performing these binary operations, a defined procedure and mathematical arithmetic is performed on not a number (nan) values of the cellular automata grid so as to be able to not have undefined hexagonal automata grid cell evaluations.

For any binary operator, there is a left side and a right side value and can be represented as

(Left Side) Operator (Right Side)
(LS) O (RS)

(nan) O (nan) = nan

logic AND

(nan) AND (RS) = RS
(LS) AND (nan) = LS

logic OR

(nan) OR (RS) = RS
(LS) OR (nan) = LS

logic EQUAL

(nan) EQUAL (RS) = false
(LS) EQUAL (nan) = false

comparison operator CO !=, <, >, <=, >=

(nan) CO (RS) = false
(LS) CO (nan) = false

Math binary operator BO(+,-)

(nan) BO(+,-) (RS) = RS

(LS) BO(+,-) (nan) = LS

Math binary operator BO(*,/, %)

(nan) BO(*,/, %) (RS) = 0

(LS) BO(*,/, %) (nan) = 0

Basically treating not a number as a non or incompatible entity that is to be ignored or renders the operation non sensible can be thought of in this way where oranges represents not a number and apples a number where one wants to find the mathematical result of the number or of comparison of apples.

How many apples are there in 10 oranges?

What is the addition of three apples to 10 oranges ?

If (5 apples and 10 oranges) = ?

This is how the derivation of not a number mathematics described above came from.

7.2.3 Parser algorithm logic_parser_class

The logic_parser_class (**logic_parser.h**) creates and manages the parser tree that is used to define the cellular automata model rules and perform the task of testing of those rules for each hexagonal automata grid cell to enable the cellular automata iteration to take place

The parser defined and used is in essence a very simple and limited top-down parser and for any rule defined as a string of text, the parser reads that text left to right one character at a time and creates a list of tokens that is to be submitted to a main expression parsing function which constructs the parser tree.

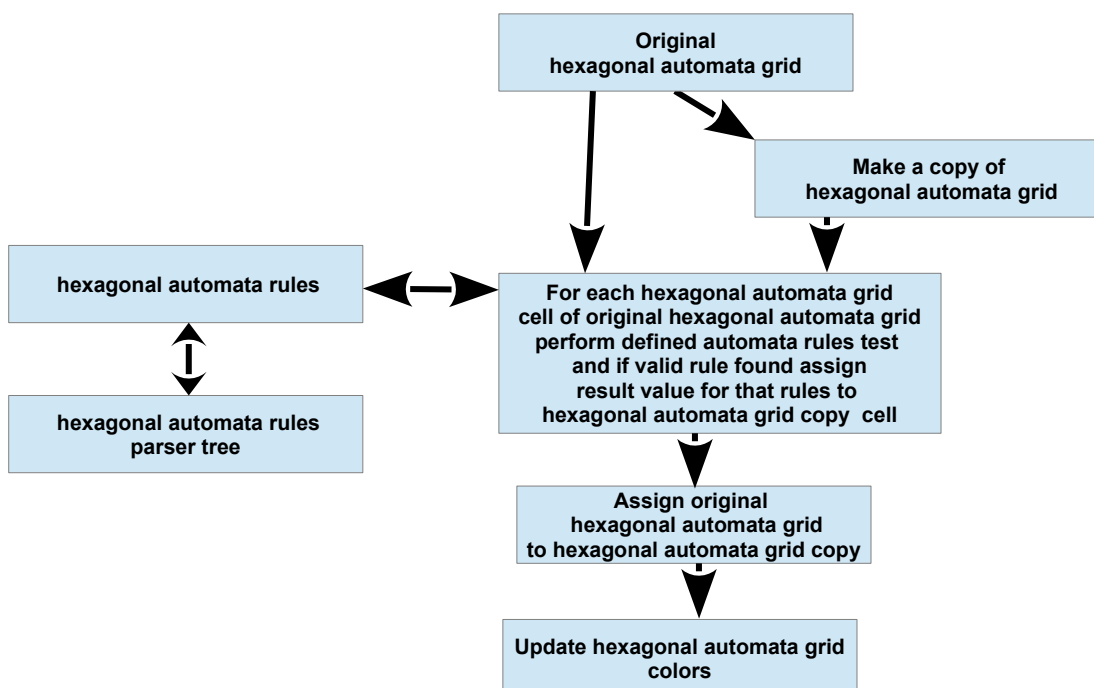
The resultant parser tree is a binary tree structure of the structure parser_base_node datatype that has all of the appropriate assignments of that datatype performed to model the expression that has been defined as a string of text.

Algorithm : Best to examine the code and comments in **logic_parser.h** to get a grasp how the parser works as to describe it here would repeat much of the code.

Evaluation of the parser tree is performed by the evaluate_parse_tree function that exists within the **logic_parser_class**. The parser tree is recursively traversed in a post order sequence to generate the end result. As each operator function is performed, the resultant value is stored in the operator node value variable which is used in the next evaluation operation above it.

7.3 Cellular Automata Iteration process

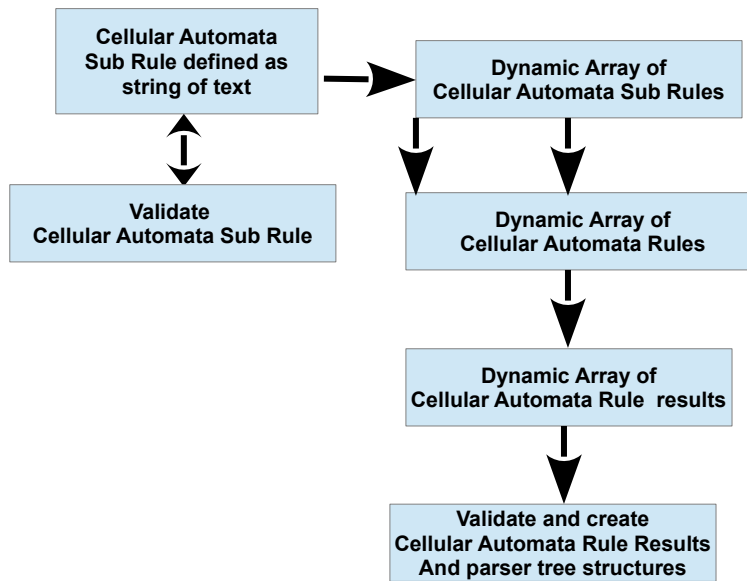
Each iteration step of cellular automata iteration process



Repeat this step process for all cellular automata iteration steps.

8 Cellular Automata Iteration Rules

8.1 Basic cellular automata rule construction and definition of cellular automata rule iteration rule results



9 Random Number Generation

random_engine_class

random_engine.h

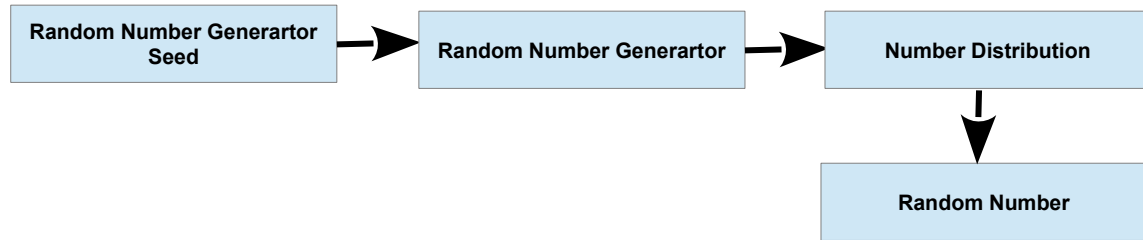


Fig 9.1

9.1 Random number generator seed

Types : User defined - User entry
 Machine defined - Time based
 - Random device

9.2 Random number generator

Types : default_engine, minstd_rand, minstd_rand0, mt19937, mt19937_64, ranlux24_base, ranlux48_base, ranlux24, ranlux48, knuth_b

All random number generators accept as a seed an integer value and thus entry provided by **9.1** is an integer value.

9.3 Number distribution

Types : Uniform_int, Uniform_real, bernoulli, binomial, geometric, negative_binomial, poisson, exponential, gamma, weibull, extreme_value, normal, lognormal, chi_squared, cauchy, fisher_f, student_t

All number generators must accept as a constructor input any one of the random number generators in **9.2**. Therefore for each number distribution type, there are 10 possible combinations of random number generator giving a total of 170 generator-distribution combinations.

Each distribution has different number and type of input parameters to define their distribution patterns and generates different output number data types.

Table 9.1 Number input parameters

One	Two
Bernoulli	Uniform_int
geometric	Uniform_real
poisson	binomial
exponential	negative_binomial
chi_squared	gamma
student_t	weibull
	extreme_value
	normal
	lognormal
	cauchy
	fisher_f

Table 9.2 Output number datatype

Boolean	Integer	Real
Bernoulli	Uniform_int	Uniform_real
	binomial	exponential
	geometric	gamma
	negative_binomial	weibull
	poisson	extreme_value
		normal
		lognormal
		chi_squared
		cauchy,
		fisher_f
		student_t

9.4 Generating a random number

Generating a random number using the standard C++ random library for all cases is in the form as illustrated in **Fig 9.1**.

In pseudo code this is for all cases

1. `random_number_seed = integer_number;`
2. `std::<random_number_generator_type> random_number_generator(random_number_seed);`
3. `std::<random_number_distribution_type> random_number_distribution(distribution parameters);`
4. `<output_data_type> random_number = random_number_distribution(random_number_generator);`

where

`random_number_generator_type` of line 2 is one of the random number generator types in **9.2**.

`random_number_distribution_type` of line 3 is one of the number distribution types in **9.3**.

The distribution parameters can be either a one or two input parameter values depending on which random distribution type is being used.

9.4.1 Base class

Thus to design a random number generator class that does not have 170 functions to cater for each possible combination, need to create base classes or structures that act as a basis to define higher level classes or structures and act as generalised inputs to functions to process and create the random number.

The random seed number used by all random number generator types is by definition of an unsigned long long datatype.

The C++ number distribution types all accept all random number generator types no matter what random number generator type is used to define the `random_number_generator(random_number_seed)` variable in line 2 as a parameter for input as given by line 4.

What this means is that a data structure of all the random number generators is defined, and upon the user choosing which random number generator to use, have that random number generator initiated to be executed and assigned to be used as a parameter in a selected number distribution type to generate a random number.

That is similar to line 4.

Random number = `selected_random_number_distribution(selected_random_number_generator(seed))`

9.4 .2

Such a function to perform this task would have the `selected_random_number_distribution` defined as a datatype, and `selected_random_number_generator(seed)` as a function that performs the random number generation.

Therefore the random_number base class is of a form

```
template <class random_number_distribution_type, typename return_value_type>
class random_number_generation_base_class
```

9.4.3

Before any random number distribution type can be used to return a random number, it has to be defined by one or two parameters relevant to that particular distribution.

This means that there needs to be two class types for the number distribution, one for number distributions with one input parameter, and another for two input parameters, and both inheriting the base class outlined in section **9.4.1**.

Because not all of the number distribution types are defined by parameters of one single datatype, a further complication is introduced that requires all the different parameter data types are taken into account. I.e boolean, integer and real or float.

What this means is that within each number distribution class, each parameter data type needs to be taken into account and the number distribution defined according to the datatype of the parameter(s) being used.

9.5 Putting it all together

9.5.1 Define random number values as a C++ union structure

The data types of the random numbers returned from, and used as input parameters to the number distribution types are not of a constant datatype, but are of datatype boolean, integer and double. Therefore to reduce the number of combinations of random number functions using the C++ random library, a C++ union of these data types can be created and used to store the number distribution parameter(s) or returned value in one convenient variable. This would mean that any use of such a data storage variable would need a means to indicate what datatype is being stored such as an enumeration class.

```
union distribution_parameter_value {  
    int integer_p;  
    float float_p;  
    bool bool_p;  
};
```

```
enum class distribution_parameter_data_type_enum { Integer, Float, Boolean };
```

9.5.2 Number distribution input parameters

Create a C++ structure to store the parameter attributes required to be input into a number generator type

Attributes that need to be defined

1. Value of parameter
2. Datatype of parameter value

which can be defined as

```
struct distribution_parameter_data_struct_type {  
    distribution_parameter_data_struct_type() {  
        value.float_p = 0.0f;  
    }  
  
    std::string parameter_text = "parameter";  
    distribution_parameter_data_type_enum data_type = distribution_parameter_data_type_enum::Float;  
    distribution_parameter_value value;  
};
```

Create a C++ structure to define and store the number distribution input parameter attributes to be used within a number distribution type.

Attributes that need to be defined

1. Distribution type or engine that the parameter(s) is to be input into
2. Type of parameter input related to number of number distribution input parameter(s), single or dual
3. Parameter input data structure for two parameters

which can be defined as

```
struct distribution_parameter_input_struct_type {  
    distribution_category_enum distribution_category = distribution_category_enum::undefined;  
    distribution_engine_enum distribution_engine = distribution_engine_enum::uniform_int;  
    distribution_parameter_input_type_enum input_type = distribution_parameter_input_type_enum::dual;  
    distribution_parameter_data_type_enum output_data_type = distribution_parameter_data_type_enum::integer;  
  
    distribution_parameter_data_struct_type parameter_1;  
    distribution_parameter_data_struct_type parameter_2;  
};
```

where have

```
enum class distribution_engine_enum { Uniform_int, Uniform_real, bernoulli, binomial, geometric, negative_binomial,  
    poisson, exponential, gamma, weibull, extreme_value, normal, lognormal, chi_sqaured, cauchy, fisher_f, student_t };
```

```
enum class distribution_category_enum { Uniform, Bernouli_related, Rate_based, Normal_related, Undefined };
```

9.5.3 Random Number generator structure type

Create a C++ structure to define the random number generator to be used and initialise it.

Attributes that need to be defined

1. The random number generator type to use
2. Random number generator seed value
3. Function to perform the generation of random numbers

4. Since 3 requires a random number generator to be defined as a variable, have every random number generator defined as a unique identified variable.

Many hours was wasted in development to try and have attribute 4 defined as a C++ template, but found that there was no way to do this without generating compiler errors or crashing the application upon successful compilation.

which can be defined as

```
struct random_number_generator_struct_type {
    number_generation_engine_enum number_generation_engine = number_generation_engine_enum::default_engine;

    bool use_default_seed = true;
    unsigned long long seed_value = 0;

    std::default_random_engine default_random_engine;
    std::minstd_rand minstd_rand_engine;
    //      V              V
    //    etc:              etc:

    void define_random_generator_engine_to_use() {
        switch (number_generation_engine) {
            case number_generation_engine_enum::default_engine : { default_random_engine.seed(seed_value); break;}
            case number_generation_engine_enum::minstd_rand    : { minstd_rand_engine.seed(seed_value); break;}
            //      V              V
            //    etc:              etc:
        }
    }
};
```

Full listing in random_engine.h

where

```
enum class number_generation_engine_enum { default_engine, minstd_rand, minstd_rand0, mt19937, mt19937_64,
ranlux24_base, ranlux48_base, ranlux24, ranlux48, knuth_b };
```

9.5.4 Random Number generation base class

With all of the structures and enumerated data types defined in section 9.5.1 to 9.5.3, have the ingredients to create the C++ random number generation base class that can be used to generate random number for all number distribution types that are defined, and the random number returned value as a C++ template in the form.

```
random_number_base_class <number_distribution_type, return_value_data_type> number_distribution;
```

Such a base class can be defined as

```
template <class random_number_distribution_type, typename return_value_type>
class random_number_generation_base_class {
public:
    random_number_generation_base_class() {}
    ~random_number_generation_base_class() {}

    random_number_generator_struct_type random_number_generator;
    distribution_parameter_input_struct_type distribution_parameters;
    return_value_type generated_number_value;

    random_number_distribution_type random_number_distribution;

    void define_random_number_generation(random_number_generator_struct_type rn_generator,
distribution_parameter_input_struct_type random_distribution_input_parameters) {
        random_number_generator = random_number_generator;
        distribution_parameters = random_distribution_input_parameters;

        define_distribution_range();
    }

    void define_distribution_range() {
        if (distribution_parameters.input_type == distribution_parameter_input_type_enum::single) {
            define_single_distribution_range();
        } else {
            if (distribution_parameters.input_type == distribution_parameter_input_type_enum::dual) {
                define_dual_distribution_range();
            }
        }
    }
};
```

```

    // do nothing where no distribution range required
}

return_value_type generate_random_number() {
    switch (random_number_generator.number_generation_engine) {
        case number_generation_engine_enum::default_engine : { generated_number_value =
            random_number_distribution(random_number_generator.default_random_engine); break; }
        case number_generation_engine_enum::minstd_rand : { generated_number_value =
            random_number_distribution(random_number_generator.minstd_rand_engine); break; }
        //
        //
        etc:
        etc:
    }

    return generated_number_value;
}
};

```

Full listing in random_engine.h

9.5.5 Random Number generation initialisation classes

Before any random number distribution type can be called to generate a random number, the distribution type needs to be initialised with input parameters. As outlined in section 9.4.3 and 9.5.2, depending upon the the number distribution type, it can have one or two parameter inputs of one of three data types. Therefore two classes of number distribution initialisation are required, both catering for the different parameter data types. These classes can utilise the C++ template using the distribution type to perform the initialisation in the form

```

random_number_generation_1_range_class<random_number_distribution_type, return_number_type> number_gen_1;
random_number_generation_2_range_class<random_number_distribution_type, return_number_type> number_gen_2;

```

Such a base class are defined as

```

template <class random_number_distribution_type, typename return_value_type>
class random_number_generation_1_range_class : public
random_number_generation_base_class<random_number_distribution_type, return_value_type> {
public:
    random_number_generation_1_range_class() {}
    ~random_number_generation_1_range_class() {}
    using RNGB = random_number_generation_base_class<random_number_distribution_type, return_value_type>;

    void define_single_distribution_range() override {
        switch (RNGB::distribution_parameters.parameter_1.data_type) {
            case distribution_parameter_data_type_enum::Integer: { RNGB::random_number_distribution =
random_number_distribution_type(RNGB::distribution_parameters.parameter_1.value.integer_p); break; }
            case distribution_parameter_data_type_enum::Float: { RNGB::random_number_distribution =
random_number_distribution_type(RNGB::distribution_parameters.parameter_1.value.float_p); break; }
            case distribution_parameter_data_type_enum::Boolean: { RNGB::random_number_distribution =
random_number_distribution_type(RNGB::distribution_parameters.parameter_1.value.bool_p); break; }
        }
    }
};

template <class random_number_distribution_type, typename return_value_type>
class random_number_generation_2_range_class : public
random_number_generation_base_class<random_number_distribution_type, return_value_type> {
public:
    random_number_generation_2_range_class() {}
    ~random_number_generation_2_range_class() {}
    using RNGB = random_number_generation_base_class<random_number_distribution_type, return_value_type>;

    virtual void define_dual_distribution_range() override {
        switch (RNGB::distribution_parameters.parameter_1.data_type) { // Following parameter testing not really needed,
but used in case have distributions parameter values using different data types in future
            case distribution_parameter_data_type_enum::Integer: {
                switch (RNGB::distribution_parameters.parameter_2.data_type) {
                    case distribution_parameter_data_type_enum::Integer : { RNGB::random_number_distribution =
random_number_distribution_type(RNGB::distribution_parameters.parameter_1.value.integer_p,
RNGB::distribution_parameters.parameter_2.value.integer_p); break;}
                    case distribution_parameter_data_type_enum::Float : { RNGB::random_number_distribution =
random_number_distribution_type(RNGB::distribution_parameters.parameter_1.value.integer_p,
RNGB::distribution_parameters.parameter_2.value.float_p); break;}
                    case distribution_parameter_data_type_enum::Boolean : { RNGB::random_number_distribution =
random_number_distribution_type(RNGB::distribution_parameters.parameter_1.value.integer_p,
RNGB::distribution_parameters.parameter_2.value.bool_p); break;}
                }
                break;}
            //
            //
        }
    }
};

```

```

        case distribution_parameter_data_type_enum::Float: {
            switch (RNGB::distribution_parameters.parameter_2.data_type) {
                case distribution_parameter_data_type_enum::Integer : { RNGB::random_number_distribution =
random_number_distribution_type(RNGB::distribution_parameters.parameter_1.value.float_p,
RNGB::distribution_parameters.parameter_2.value.integer_p); break;}
                case distribution_parameter_data_type_enum::Float : { RNGB::random_number_distribution =
random_number_distribution_type(RNGB::distribution_parameters.parameter_1.value.float_p,
RNGB::distribution_parameters.parameter_2.value.float_p); break;}
                case distribution_parameter_data_type_enum::Boolean : { RNGB::random_number_distribution =
random_number_distribution_type(RNGB::distribution_parameters.parameter_1.value.float_p,
RNGB::distribution_parameters.parameter_2.value.bool_p); break;}
            }
            break; }
        case distribution_parameter_data_type_enum::Boolean: {
            switch (RNGB::distribution_parameters.parameter_2.data_type) {
                case distribution_parameter_data_type_enum::Integer : { RNGB::random_number_distribution =
random_number_distribution_type(RNGB::distribution_parameters.parameter_1.value.bool_p,
RNGB::distribution_parameters.parameter_2.value.integer_p); break;}
                case distribution_parameter_data_type_enum::Float : { RNGB::random_number_distribution =
random_number_distribution_type(RNGB::distribution_parameters.parameter_1.value.bool_p,
RNGB::distribution_parameters.parameter_2.value.float_p); break;}
                case distribution_parameter_data_type_enum::Boolean : { RNGB::random_number_distribution =
random_number_distribution_type(RNGB::distribution_parameters.parameter_1.value.bool_p,
RNGB::distribution_parameters.parameter_2.value.bool_p); break;}
            }
            break; }
    }
}
};

```

listing in random_engine.h

9.5.6 Random Number generation Procedure

With all of the structures, base class and classes defined in sections 9.5.1 to 9.5.5 the procedure to generate a random number is

1. Select the random generator to use as a selection variable and store it in a random number structure type of **9.5.3**
2. Define the random number generation seed value and store it in a random number structure type of **9.5.3**
3. Generate the random numbers using the selection and seed store in the random number structure type of **9.5.3**.
4. Select the number distribution to use and display input parameters to use to initialise it.
5. Define the input parameter values to initialise the selected number distribution type and store the values in a structure type of **9.5.2** struct distribution_parameter_input_struct_type.
6. Perform the function define_random_number_generation of the base class **9.5.4** to define the random number generator and distribution input parameters to use to generate a random number.
7. Generate the random number for the selected number distribution and assign it to the generated number value of the random_number_generation_base_class
8. Assign the value of the generated random number distribution in step 7 to a hexagonal automata grid cell.

Steps 7-8 would involve the generation of a random number for each hexagonal automata grid cell which is performed in a for loop.

This random number generation procedure would unavoidably need to be performed for each number distribution type, and therefore there would be required to be 17 functions to generate the random numbers for the hexagonal automata grid. This is better than the 170+ permutations that otherwise would be needed if each permutation of random number generator per number distribution was implemented.

Each of these function would look very similar and identical apart from the number distribution. In pseudo code this function would look like

```

void distribution_function_name(){
    for (size_t i = 0; i < HGBC::hex_grid.size(); i++) {
        random_engine.<Distribution_type_name>.generate_random_number();

        T rn = (T)random_engine.<Distribution_type_name>.generated_number_value;

        hex_grid[i] = rn;
    }
}

```

17 such functions are defined in the hex_grid class within the hex_grid.h code file with the name of format <distribution_type>_distribution_hex_grid_random_generation.

10 Export/Import of data files

All the exporting and importing of data files use the same principle of writing to and reading from a text file one line at a time, and depending upon the type of data being written or read, will expect the data to follow a certain order given for that type of data.

10.1 File Format

The text file format for all files follows the same principle of having data of a given type defined as a block of data between two lines of text indicating a beginning of a data block and an end of a data block. These begin and end data block text can be of any defined text, be it a single character or a string of characters

10.1.1 In pseudo like code this would be of form

```
<Begin data block>
data value 1;
data value 2;
:
datavalue n;
<End data block>
```

Such a data format are for flat C++ data structures that have no sub structure data types

10.1.2 Multiple such data blocks can exist within a data file and even have an embedded form of

```
<Begin data blockA>
data value 1A;
data value 2A;
:
datavalue nA;
<Begin data blockB1>
data value 1B1;
data value 2B1;
:
datavalue nB1;
<End data blockB1>
<End data blockA>
```

Such a data format are for C++ data structures that have one sub structure data types such as a vector array or another C++ structure datatype.

10.1.3 Can have multiple embedded data blocks for structures of data that have multiple entries of a same type.

```
<Begin data blockA>
data value 1A;
data value 2A;
:
datavalue nA;
<Begin data blockB1>
data value 1B1;
data value 2B1;
:
datavalue nB1;
<End data blockB1>
<Begin data blockB1>
data value 1B1;
data value 2B1;
:
datavalue nB1;
<End data blockB1>
<End data blockA>
```

Such a data format are for C++ data structures that have more than one sub structure data types such as vector arrays and/or sub structure data types.

10.1.4 Can have one or more embedded data blocks within an embedded data block similar to C++ code data blocks.

In General, can consider the principle of the data files being saved as following the same as C++ code blocks. In addition to the individual data block flags for each type of data, there is one overall data block flag

pair for the entire data file. This is so that in the importation of data file, the importation file function can have a point to recognise and test that the entire data file has been read correctly and that the end of data has been reached. That is the entire data file format for all data files can be summarised as

```
<Begin file data block>
<Data block A>
:
<Data block n>
<End file data block>
```

10.2 Exporting/importing data files

Each type of data being exported or imported follows a set given format for that type of data following the principles outlined in 10.1.

A given begin/end data block pair of flags is defined for a particular set of data which is written before and after that data set. The data set being written that may be of data of different data types is in a given order that when imported must be read in the same order and defined as the same datatype as exported.

Data that is of a repetitive nature can invoke a write/read recursive function in some cases.

The general procedure to export data of any kind to a disc file is

- 1 Define path name of file to export to.
- 2 Open file to write to
- 3 Write begin file data block flag
- 4 For each data structure to export
 - 4.1 Write data structure begin data block flag
 - 4.2 write data value
 - 4.2.1 For each embedded data structure
 - 4.2.1.1 Write embedded data structure begin data block flag
 - 4.2.1.2 Write data value
 - 4.2.1.3 Write embedded data structure end data block flag
 - 4.3 Write data structure end data block flag
- 5 Write end file data block flag
- 6 Close file to write to.

Multiple levels of embedding of data may exist within each embedded level of 4.2 and thus have a recursive fractal like nature of data of 4.2.1 within 4.2.1.

The general procedure to import data of any kind to a disc file is

- 1 Define path name of file to import data from
- 2 Open file to read from
- 3 Read entire file into a string buffer
- 4 Close data file reading from
- 5 Define a vector of character strings for each line of text in 3
- 6 For first string vector index entry in 4
 - 6.1 Define as a line of text
 - 6.2 Test if have begin file data block flag : if not send error message and exit
 - 6.3 Read next vector index entry in 4
 - 6.4 Test if have begin data block flag : if not send error message and exit
 - 6.4.1 If have embedded data block test for begin data block flag : if not send error message and exit
 - 6.4.2 Read each next vector index entry in 4 that equates to the data exported
 - 6.4.3 Read next vector entry in 4.
 - 6.4.4 Test If have correct end data block flag : if not send error message and exit
 - 6.5 Read next vector entry in 4.
 - 6.6 Test If have correct end data block flag : if not send error message and exit
- 7 Read next vector entry in 4
- 8 Test if have correct end data block flag : if not send error message and exit

Even if step 8 records an error, the data should have been imported correctly and no need to cancel the data imported.

Step 6 must read the data in the correct order as exported and test performed that the data is converted from text to numeric where required to avoid crashes.